

TypeScript 기본

TypeScript

- JavaScript의 슈퍼셋 프로그래밍 언어
- 2014년 4월 마이크로소프트에서 출시
- 정적 타입(Static Type)를 지원
- 컴파일 과정을 통해서 JavaScript로 변환
- 컴파일 과정에서 런타임 에러를 사전 방지
- 자동완성 및 생산성 향상
- 코드의 가독성 향상



- npm(Node Package Manager)은 JavaScript 생태계의 최대의 소프트웨어 레지스트리 패키지 관리자
- npm registry는 npm으로 관리되는 모든 패키지의 저장소
- Package.json, Package-lock.json으로 의존성 관리
- npm init 으로 초기화
- npm 명령을 사용해서 필요한 라이브러리를 설치
 - npm i express, npm i -g typescript

Package.json

항목	내용
package name	패키지 이름으로 프로젝트 혹은 현재 애플리케이션 이름을 말한다.
version	패키지, 프로젝트, 애플리케이션 버전 면을 말한다.
description	패키지, 프로젝트, 애플리케이션의 설명을 말한다.
entry point	프로젝트에서 가장 먼저 실행되는 자바스크립트 실행파일로 웹 서버의 경우 index.js 혹은 app.js 로 지정해 준다.
test command	코드 테스트 시에 입력해 줄 명령어로, 사용하지 않을 경우 빈 값으로 지정된다.
git repository	코드를 저장해둔 깃(Git) 저장소 주소로 사용하지 않을 경우 빈 값으로 지정된다.
keywords	패키지, 프로젝트, 애플리케이션의 검색을 위한 키워드
license	패키지, 프로젝트, 애플리케이션의 라이선스로 기본값은 ISC (Internert Systems Consortium)에서 허용한 자유 소프트웨어 라이선스

데이터형(Data Type)

기본 데이터형(Data Type)

- number: 숫자
- string : 문자열
- boolean : true, false
- object : 객체타입
- array : 배열
- tuple : 각 요소의 타입과 순서가 고정된 배열 타입
- any : 모든 타입
- null : 값이 없음
- undefined: 정의되지 않은

타입 지정

```
let num: number = 10;
const greet: string = 'Hello TypeScript';
const isOn: boolean = true;
const person: object = { name: '손흥민', age: 10 };
const numbers: number[] = [1, 2, 3, 4, 5, 6];
const bts: string[] = ['진', '슈가', '제이홉'];
const item: [string, number, boolean]
    = ['손흥민', 30, true];
let name: any = '손흥민';
name = 1;
// num = 'hello'; //타입 불일치 에러 발생
let empty: null = null;
let notAssigned: undefined = undefined;
```


함수의 매개변수와 반환값

```
function add(num1: number, num2: number): number {  
    return num1 + num2;  
}  
  
const result = add(5, 4);  
const result1 = add('1', '2'); //매개변수의 타입이 불일치  
const result2: string = add(4, 5); //반환타입과 상수타입이 불일치  
const result3: number = add(4);
```


함수의 매개변수와 반환값

```
function sayHello(name: string): string {  
    return `Hello ${name}`;  
}
```

```
let greeting = sayHello('RM');  
console.log(greeting);
```

```
function sayHi(name: string): void { // 반환값 없음  
    console.log(`Hi ${name}`);  
}
```

```
const hi = sayHi('손흥민');  
console.log(hi);
```

함수형 매개변수

```
function sayHello(name: string): string {  
    return `Hello ${name}`;  
}
```

```
function greeting(name: string,  
                  callback: (name: string) => string): string  
{  
    return callback(name);  
}
```

```
const result = greeting('손흥민', sayHello);  
const result1 = greeting('이강인', (name: string) => `Hello  
${name}`);  
console.log(result, result1);
```

형 추론

```
let name = '손흥민'
let age = 30
let isOn = true
const person1 = { name: '손흥민', age: 10 };
const bts = ['진', '슈가', '제이홉'];
console.log(typeof name);      // string
console.log(typeof age);       // number
console.log(typeof isOn);      // boolean
console.log(typeof person1);   // object
console.log(typeof bts);       // object
name = 1 //타입 에러
age = '1' //타입에러
person1 = {name: 'RM'}
```

형 추론

```
function sayHello(name: string): string {  
    return `Hello ${name}`;  
}  
let greeting = sayHello('RM');  
function sayHello1(name: string) { //string으로 형추론  
    return `Hello ${name}`;  
}  
function sayHello2(name: string) { //void로 형추론  
    console.log(`Hello ${name}`);  
    return;  
}  
function sayHello3(name: string) { //void로 형추론  
    console.log(`Hello ${name}`);  
}
```

union type

```
let num: number | string = 10;  
num = '10';  
console.log(typeof num);
```

```
function getAge(age: number | string): string {  
    return '저의 나이는 ' + age;  
}
```

```
console.log(getAge(20));  
console.log(getAge('20'));
```

Quiz-함수 정의 실습

구구단의 단을 매개변수로 전달받아
출력하는 함수를 정의하고 호출하시오.

Quiz-함수 정의 실습

점수(score)를 매개 변수로 전달받아
학점(A,B~F)을 반환하는 함수를 정의하고 여러번 호출하시오.

interface

interface

객체의 타입을 정의할 때 사용

```
const person = { name: '손흥민', age: 30 };
```

```
interface Person {  
  name: string;  
  age: number;  
}
```

```
const person1: Person = { name: '손흥민', age: 30 };
```

```
const person2: Person = { name: '손흥민' };
```

```
const person3: Person = { name: '손흥민', age: 30, team: 'LA' };
```

옵션 속성(optional)

```
interface Student {  
  name: string;  
  grade: number;  
  major?: string;  
}  
  
function studentInfo(s: Student) {  
  console.log(s.name);  
  console.log(s.grade);  
}  
  
studentInfo({ name: '손흥민', grade: 1, major: '컴공' });  
const s1: Student = { name: '손흥민', grade: 1, major: '컴공' };  
studentInfo(s1);  
const s2 = { name: '손흥민', grade: 1, major: '컴공' };  
studentInfo(s2);
```

interface 상속

```
interface Person {  
    name: string;  
    age: number;  
}
```

```
interface Student extends Person {  
    major: string;  
}
```

```
const s: Student = { name: '손흥민', age: 1, major: '컴공' };  
const s1: Student = { name: '손흥민', age: 1 }; //에러 발생
```

interface-union type

```
interface Person {  
  name: string;  
  age: number;  
}
```

```
interface Student extends Person {  
  major: string;  
}
```

```
interface Player extends Person {  
  skill: string;  
}
```

```
const p1: Student = { name: '손흥민', age: 30, major: '컴공' };  
const p2: Player = { name: '손흥민', age: 30, skill: '축구' };
```

interface-union type

union 매개변수일때는 속성이 존재하는지 검사 필요

```
function description(p: Student | Player) {  
  if ('major' in p) {  
    const info = `${p.name}, ${p.age}, ${p.major}`;  
    console.log(info);  
    return;  
  }  
  if ('skill' in p) {  
    const info = `${p.name}, ${p.age}, ${p.skill}`;  
    console.log(info);  
  }  
}  
description(p1);  
description(p2);
```

interface-Intersection

타입 2개를 합쳐서 사용

```
function info(p: Student & Player) {  
  const result = `${p.name}, ${p.major}, ${p.skill}`;  
  console.log(p.name);  
}  
  
info({ name: '손흥민', age: 30, major: '컴공', skill: '축구' });
```

interface-선언병합

같은 이름으로 interface를 정의하면 병합됨

```
interface Person {  
    name: string;  
}
```

```
interface Person {  
    age: number;  
}
```

```
interface Person {  
    name: string;  
    age: number;  
}
```


interface-객체 모델링

```
interface Maker {  
    name: string;  
    country: String;  
}
```

```
interface Car {  
    model: string;  
    maker: Maker;  
}
```


Quiz-모델링실습

[http://raw.githubusercontent.com/wapj/jsbackend/main/
movieinfo.json](http://raw.githubusercontent.com/wapj/jsbackend/main/movieinfo.json)

위의 주소에 응답 JSON을 보고 interface를 사용해서 모델링하시오.

Quiz-회원가입

JavaScript 수업에 했던 회원가입(Promise사용버전)을
TypeScript로 변환하라

type-alias

타입별칭

특정 타입에 의미론적 별칭을 붙임, 중복 불가

```
type PersonName = string;
```

```
let name: string = '신호탄';
```

```
let name1: PersonName = '국기봉';
```

```
function printAge(num: string | number) {  
    console.log(num);  
}
```

```
type Age = string | number;
```

```
function printAge1(num: Age){  
    console.log(num)  
}
```

타입별칭

```
interface Car {}  
interface Plane {}  
type Ship = {}
```

```
type Vehicle = Car | Plane;  
type Vehicle1 = Car & Plane;
```

```
type Vehicle2 = Vehicle | Plane;
```

interface vs type

```
type User = {  
  id: string;  
  name: string;  
}
```

```
interface User1 {  
  id: string;  
  name: string;  
}
```

```
let user: User;  
let user1: User1;
```

구분	interface	type
용도	객체 구조	모든 타입
확장	extends	intersection
선언병합	가능	불가능
사용범위	제한적	매우 넓음

class

TypeScript에서의 class정의

```
class Person {  
  name: string;  
  age: number;  
  constructor(name: string, age: number) {  
    this.name = name;  
    this.age = age;  
  }  
  add(n1: number, n2: number): number {  
    return n1 + n2;  
  }  
}
```

```
let p:Person = new Person('손흥민', 30)
```


접근 제한자-private

```
class Student {  
    public name: string;  
    private grade: number;  
    constructor(name: string, grade: number) {  
        this.name = name;  
        this.grade = grade;  
    }  
    public get Grade(): number { //getter  
        return this.grade;  
    }  
    public set Grade(value: number) { // setter  
        if (value < 1 || value > 5) {  
            throw new Error('학년은 1~4 사이여야 합니다.');        }  
        this.grade = value;  
    }  
}
```

접근 제한자-private

```
const person = new Person('손흥민', 30);  
const age = person.Age;  
console.log(age);  
person.Age = 20;  
console.log(person.Age);
```

접근 제한자-protected

```
class Student {  
    name: string;  
    protected grade: number;  
  
    constructor(name: string, grade: number) {  
        this.name = name;  
        this.grade = grade;  
    }  
  
    getGrade(): number {  
        return this.grade;  
    }  
}
```

접근 제한자-protected

```
class HighSchoolStudent extends Student {  
  constructor(name: string, grade: number) {  
    super(name, grade);  
  }  
  
  public printGrade() {  
    // 자식 클래스라서 접근가능  
    console.log(`학년: ${this.grade}`);  
  }  
  
  public printGrade2() {  
    console.log(`학년: ${this.getGrade()}`);  
  }  
}  
  
const s = new HighSchoolStudent('손흥민', 1)  
s.grade = 2 //error
```

Generics(제네릭)

제네릭

```
function getNumber(value: number): number {  
    return value;  
}  
function getString(value: string): string {  
    return value;  
}  
function getSimple<T>(value: T): T {  
    return value;  
}  
const result = getSimple<number>(1);  
console.log(result);  
const result1 = getSimple<string>('손흥민');  
console.log(result1);  
const result2 = getSimple(true);  
console.log(result2);
```

제네릭

```
function getBoolean(bool: boolean) {  
    return bool;  
}  
function getArray(arr: string[]) {  
    return arr;  
}  
function getObject(obj: { name: string }) {  
    return obj;  
}  
function getValue<T>(value: T): T {  
    return value;  
}  
getValue(true);  
getValue<string[]>(['1']);  
getValue({ name: '손흥민' });
```


제네릭-interface

```
interface Box<T> {  
    value: T;  
}  
  
const numberBox: Box<number> = {  
    value: 10,  
};  
  
const stringBox: Box<string> = {  
    value: 'hello',  
};
```


제네릭-interface

```
interface Pair<T, U> {  
    first: T;  
    second: U;  
}
```

```
const p1: Pair<string, number> = {  
    first: '손흥민',  
    second: 30,  
};
```

```
const p2: Pair<boolean, string> = {  
    first: true,  
    second: 'OK',  
};
```

제네릭-interface

```
interface ApiResponse<T> {  
    data: T;  
    status: number;  
}
```

```
const res: ApiResponse<string> = {  
    data: 'success',  
    status: 200,  
};
```

```
interface List<T> {  
    items: T[];  
}
```

제네릭-Type제한

//length 속성이 있는 타입만 허용

```
function printLength<T extends { length: number }>(value: T) {  
    console.log(value.length);  
}  
  
printLength('hello');  
printLength([1, 2, 3]);  
printLength({ length: 3, name: '손흥민' });  
printLength(123); //에러발생
```

제네릭-Type제한

//특정 타입만 허용

```
function logValue<T extends string | number>(value: T) {  
    console.log(value);  
}
```

```
logValue('hi');
```

```
logValue(10);
```

```
logValue(true);
```

제네릭-Type제한

//객체 속성 제한

```
function printName<T extends Person>(obj: T) {  
    console.log(obj.name);  
}
```

```
printName({ name: '손흥민' });  
printName({ name: '손흥민', age: 30 });  
printName({ age: 30 }); //에러발생
```

제네릭-Type제한

//키 제한

```
function getProperty<T, K extends keyof T>(obj: T, key: K) {  
    return obj[key];  
}
```

```
const user = { name: "손흥민", age: 30 };
```

```
getProperty(user, "name");
```

```
getProperty(user, "age");
```

```
getProperty(user, "job"); // 에러 발생
```

TODOAPP -JS

```
class TodoApp {  
  todos = [];  
  nextId = 1;  
  // 할 일 추가  
  addTodo(text) {  
    const todo = {  
      id: this.nextId++,  
      text,  
      completed: false,  
    };  
    this.todos.push(todo);  
  }  
}
```


TODOAPP -JS

// 할 일 목록 출력

```
listTodos() {  
  console.log('📌 Todo List');  
  this.todos.forEach((todo) => {  
    console.log(`[${todo.completed ? '✓' : ' '}]  
      (${todo.id}) ${todo.text}`);  
  });  
}
```

// 완료 처리

```
toggleTodo(id) {  
  const todo = this.todos.find((t) => t.id === id);  
  if (todo) {  
    todo.completed = !todo.completed;  
  }  
}
```


TODOAPP -JS

// 삭제

```
removeTodo(id) {  
  this.todos = this.todos.filter((t) => t.id !== id);  
}  
}
```

// 3. 실행 예제

```
const app = new TodoApp();  
app.addTo('TypeScript 공부하기');  
app.addTo('운동하기');  
app.listTodos();  
app.toggleTodo(1);  
app.listTodos();  
app.removeTodo(2);  
app.listTodos();
```

Quiz-TodoApp

위의 TodoApp을 TypeScript로 변환하라

Quiz-영화정보앱

JavaScript 수업에 했던 영화블로그
앱(async/await 사용버전)을 TypeScript로 변환하라

감사합니다.